Sistemas de Informação



Disciplina: Programação I

Prof. Renê Rodrigues Veloso

Herança

Mecanismo pelo qual se permite herdar características (estado e comportamento) de outra classe. É a criação de uma classe com base em outra existente.

Herança permite reusar métodos e atributos da classe herdada ou adicionar novos.

Herança: terminologias e conceitos

- **Superclasse (classe pai ou classe base):** a classe na qual as características são herdadas.
- **Subclasse (classe filha ou classe derivada):** a classe que herda e/ou estende as características de outra classe.
- **Reuso de código:** o código escrito na superclasse é comum a todas as subclasses. Classes filhas podem usar diretamente o código da classe pai.

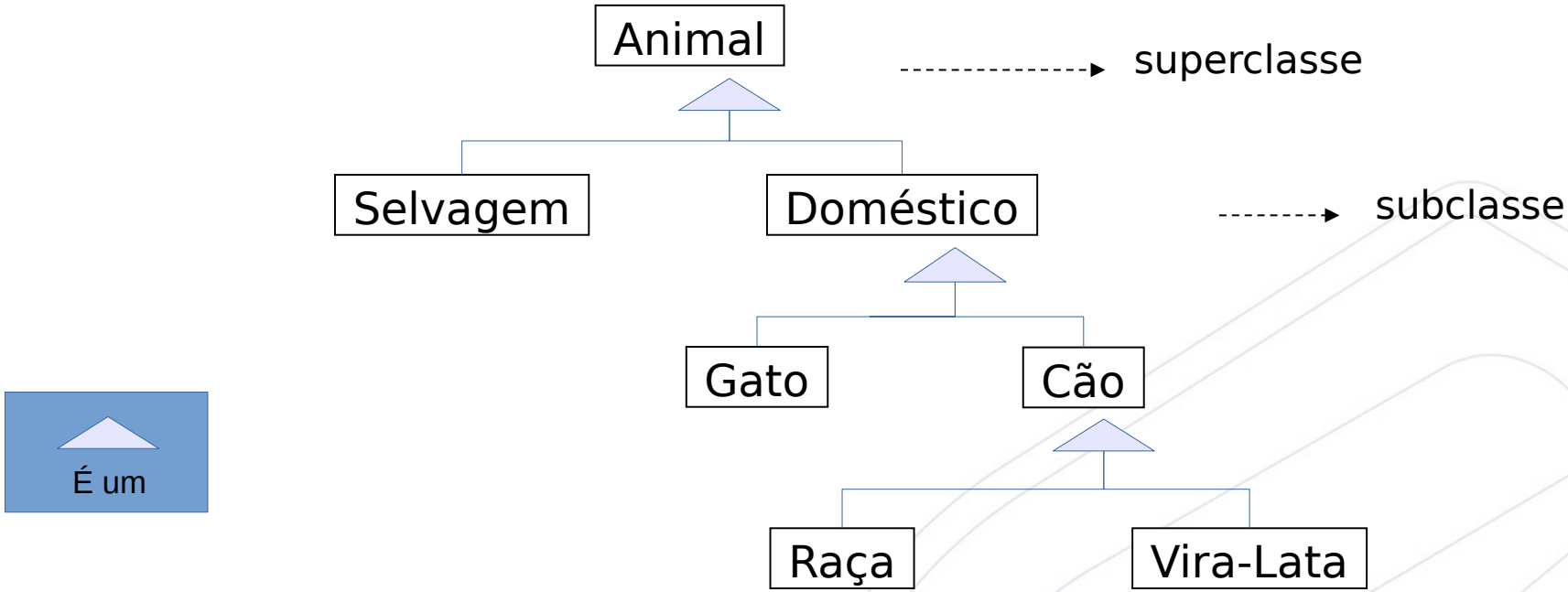
Herança: terminologias e conceitos

- **Sobreposição de métodos (*overriding*):** conceito fundamental em OO, também chamado de polimorfismo de tempo de execução (*Run Time Polymorphism*). Métodos privados ou estáticos não podem ser sobrepostos.
- **Sobrecarga de métodos (overloading):** vários métodos com o mesmo nome e assinaturas diferentes (não é um conceito exclusivo do mecanismo de herança como no caso do overriding).
- **Abstração:** um conceito na qual não precisamos fornecer todos os detalhes de implementação, mostrando apenas as funcionalidades a um usuário.

Herança

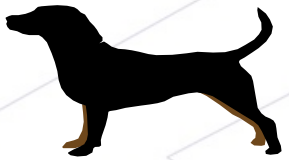
- facilita a extensibilidade
- A herança pode ser de dois tipos:
  - **Simples**: quando uma classe é subclasse de somente uma superclasse.
  - **Múltipla**: quando uma classe é subclasse de várias superclasses e conseqüentemente herda as características de cada uma delas.

Herança Simples



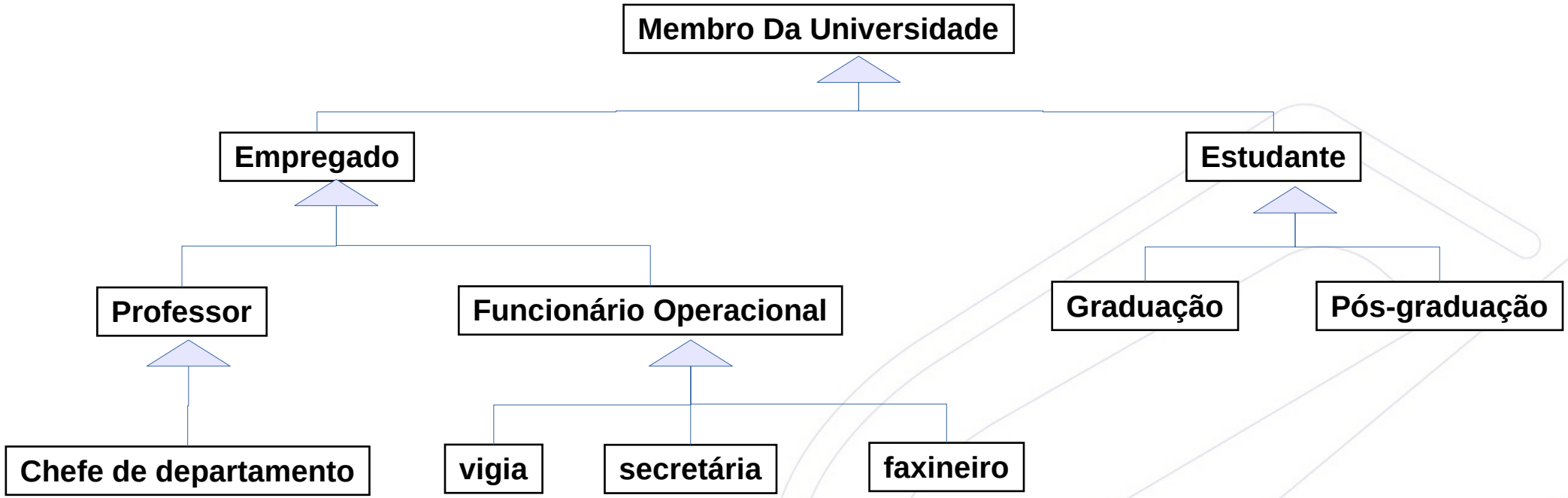
Objeto Rex

instância da classe Vira-Lata

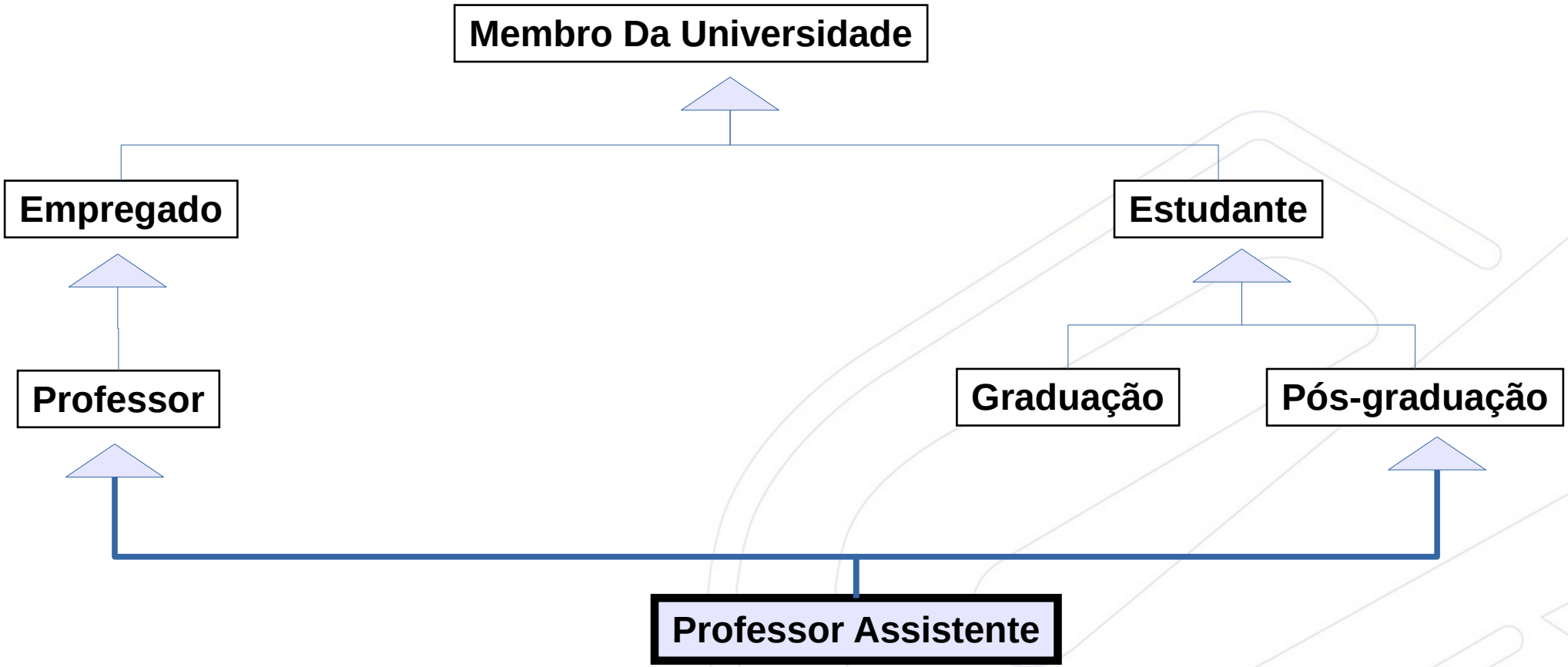


Rex

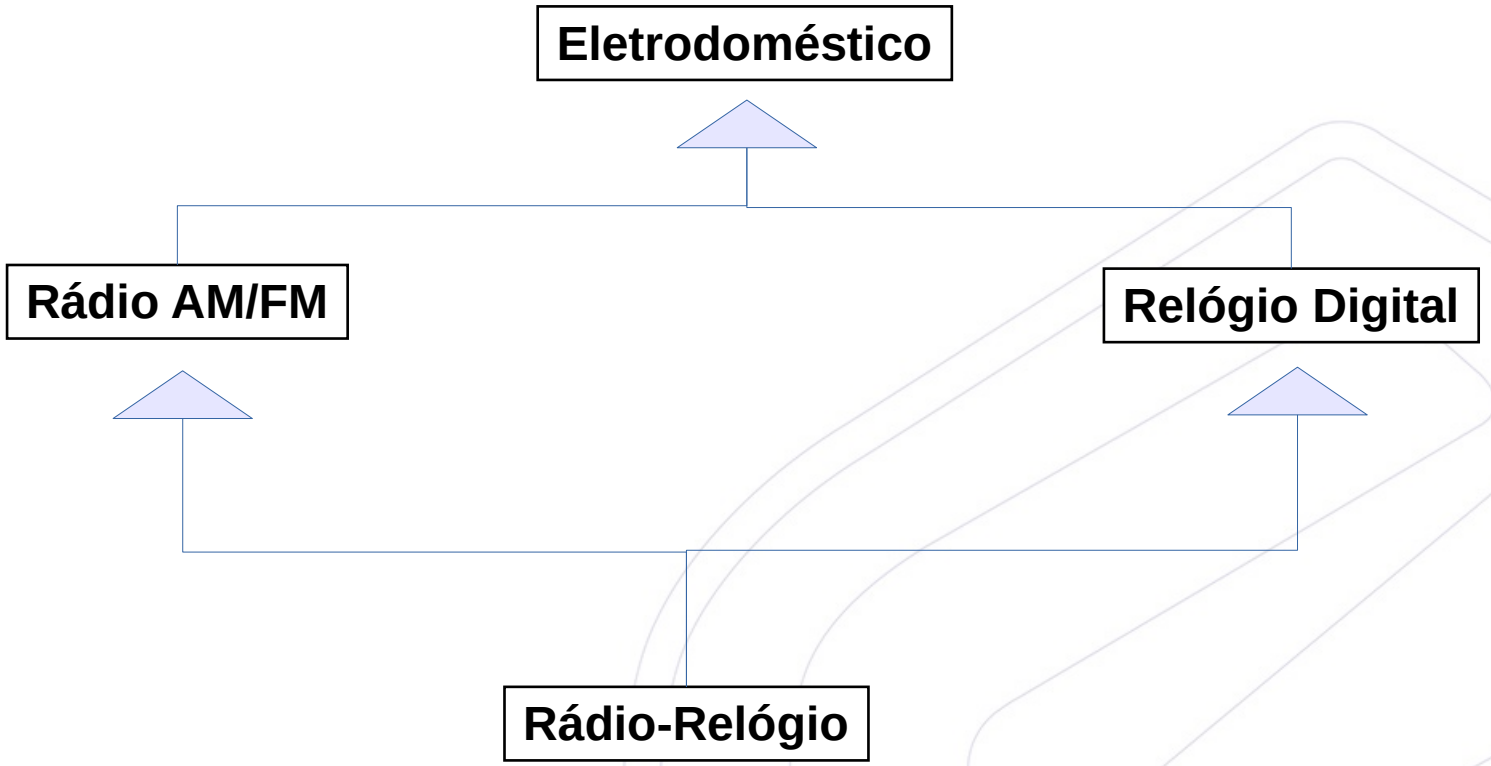
Herança Simples



Herança Múltipla

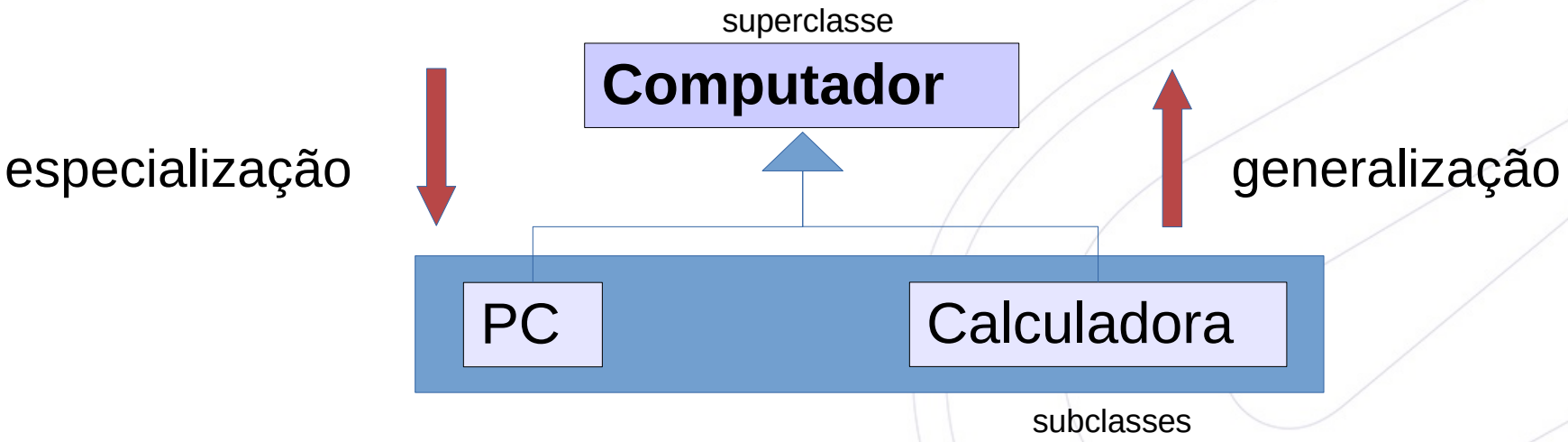


Herança Múltipla



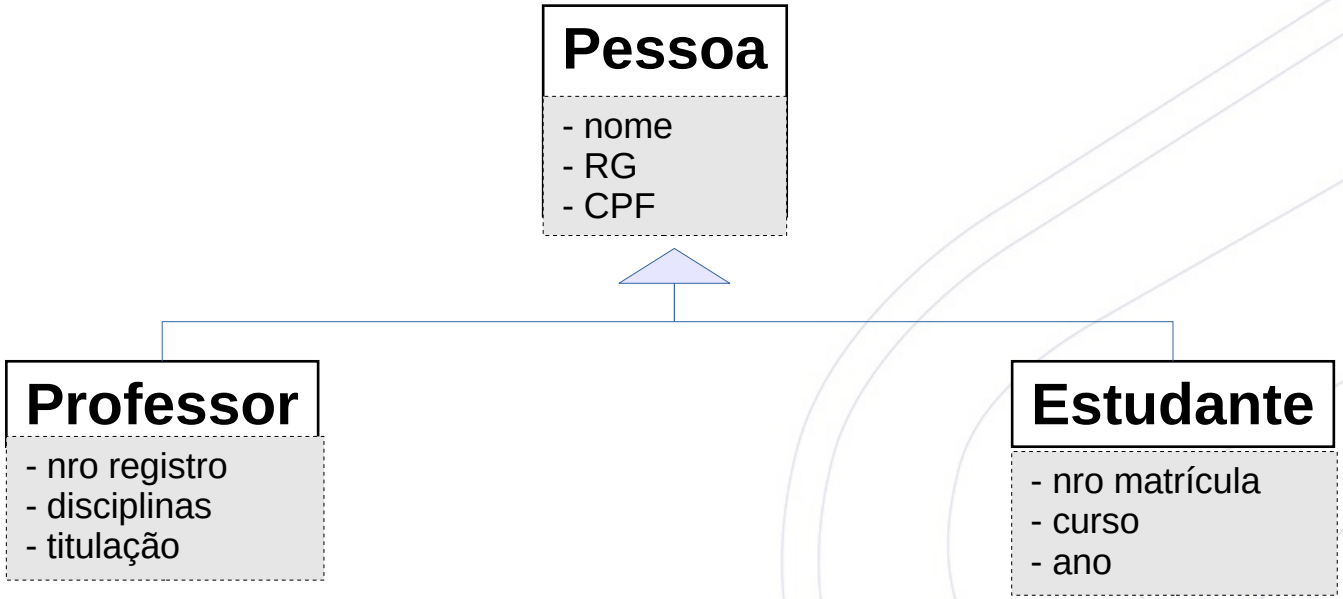
Generalização x Especialização

- O mecanismo de herança pode ser chamado de **especialização**.
- A **generalização** ocorre quando, a partir da observação de duas classes, abstraímos dela uma classe mais genérica.



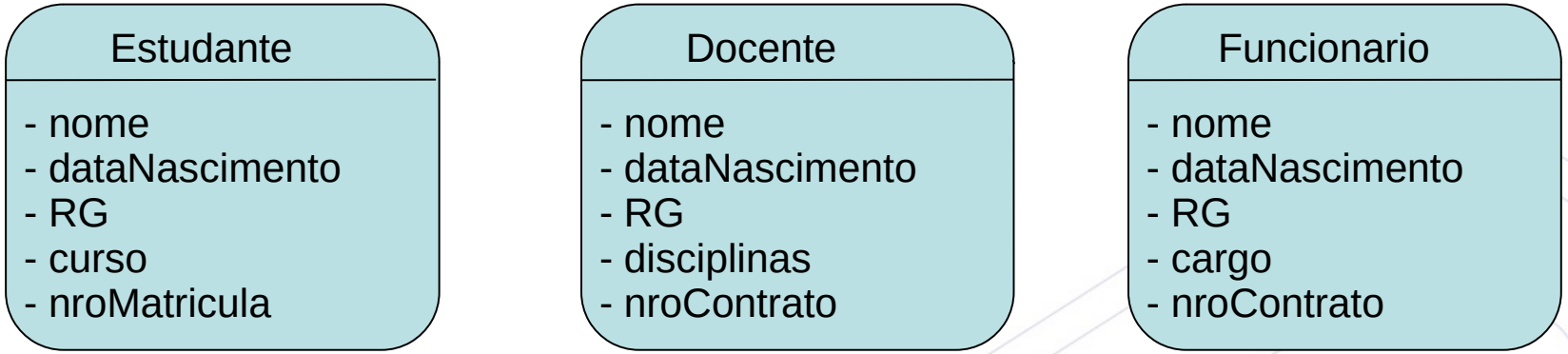
Compartilhamento de características

- O mecanismo de herança permite que os objetos envolvidos compartilhem seu estado e comportamento com as suas subclasses.



Exemplo

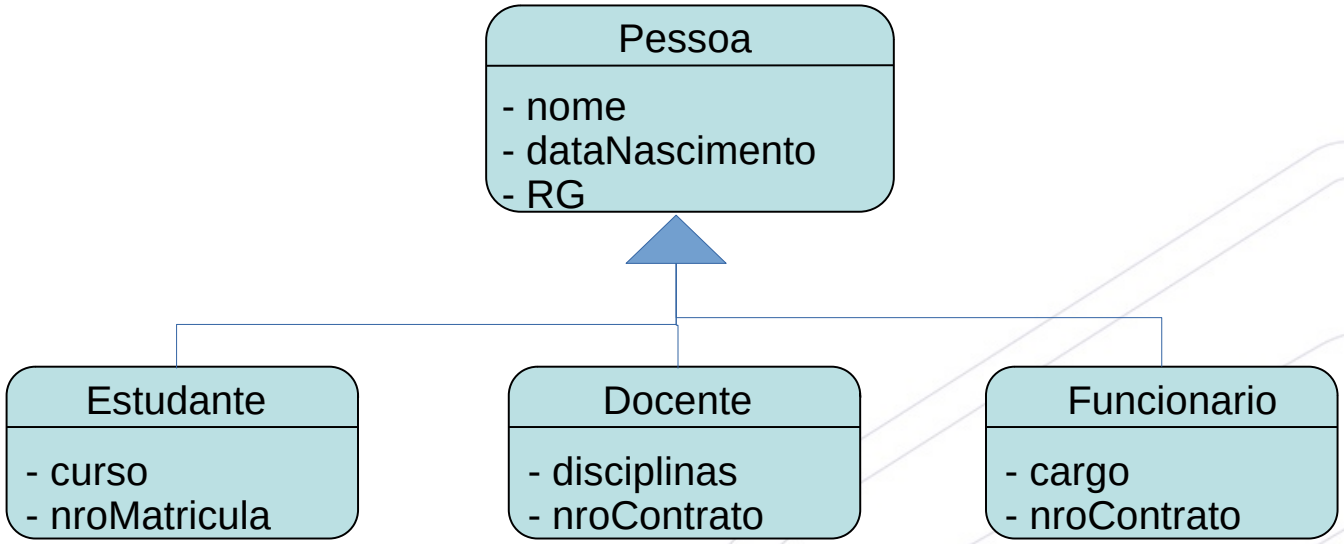
Sistema de administração universitária.



Para adicionar uma nova classe (por exemplo, Vestibulando) teríamos que repetir todos os atributos como: ***nome***, ***dataNascimento***, ***RG*** e mais o ***nroInscrição*** e o ***curso***.

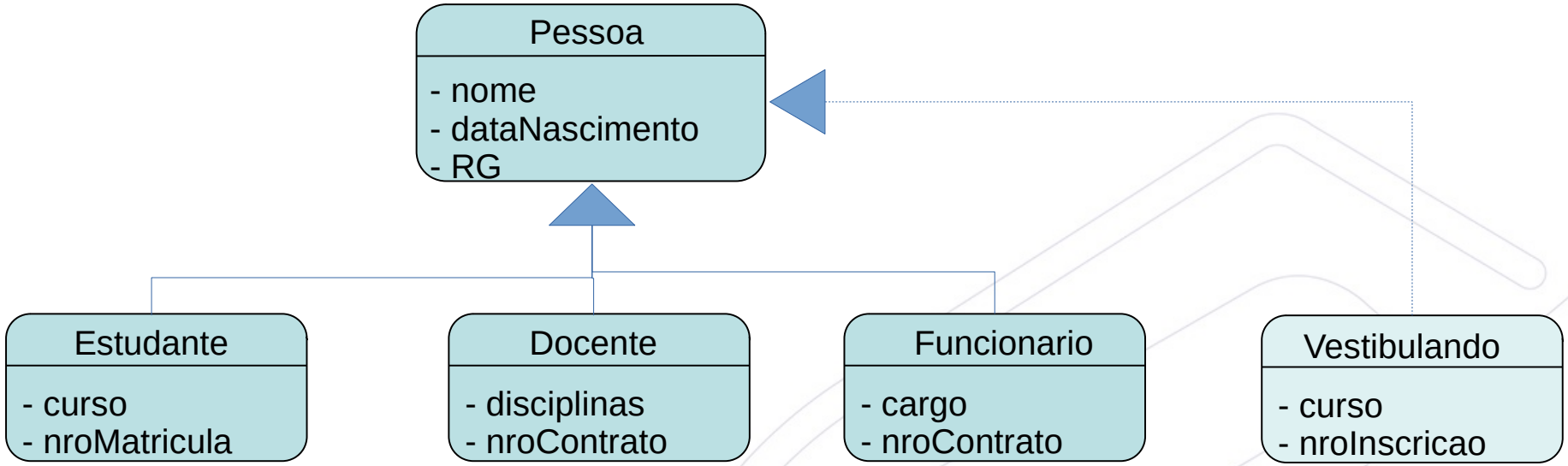
Exemplo

Sistema de administração universitária.



Exemplo

Sistema de administração universitária.



Herança em java

A palavra ***extends*** é usada para indicar a superclasse.

```
class subclasse extends superclasse
{
   //metodos and atributos
}
```

Exemplo

```
class Employee {
    int salary = 60000;
}


class Engineer extends Employee {
    int benefits = 10000;
}


class Main {
    public static void main(String args[])
    {
        Engineer E1 = new Engineer();
        System.out.println("Salario: "+ E1.salary +"Beneficios: "+ E1.benefits);
    }
}
```

Construtores em subclasses

- Quando um objeto se uma subclasse é instânciado, o construtor da superclasse deve ser chamado para fazer qualquer inicialização necessária dos seus atributos. Quando nenhum é chamado o construtor padrão da superclasse é chamado.
- Uma chamada explícita ao construtor da superclasse pode ser fornecida dentro do construtor da subclasse via operador ***super***.

```
class SuperClasse {
            SuperClasse( ) {
                ...
            }
}

class SubClasse extends SuperClasse
{
   SubClasse( )
   {
      super( );
       ...
   }
}
```

```
class SuperClasse {
            SuperClasse(int par) {
                ...
            }
}

class SubClasse extends SuperClasse
{
   SubClasse(int par1, int par2)
   {
       super ( par2 );
        ...
    }
}
```

Exemplo: com uso de sobreposição de método

```
class Bicicleta {
    public int marcha;
    public int velocidade;

    public Bicileta(int marcha, int velocidade){
        this.marcha = marcha;
        this.velocidade = velocidade;
    }

    public void freia(int decremento){ velocidade -= decremento; }

    public void acelera(int incremento){ velocidade += incremento; }

    // toString() imprime informacoes de Bicicleta
    public String toString()
    {
        return ("Marcha: " + marcha + "\nVelocidade: " + velocidade);
    }
}
```

Exemplo: com uso de sobreposição de método

```
class MountainBike extends Bicicleta {
    public int alturaAcento;

    public MountainBike(int marcha, int velocidade, int alturaAcento)
    {
        super(marcha, velocidade);
        this.alturaAcento = alturaAcento;
    }

    public void setAlturaAcento(int novoValor){ alturaAcento = novoValor; }

    // sobrepoe toString()
    // para imprimir mais informações
    @Override
    public String toString()
    {
        return (super.toString() + "\nAltura do acento: "+ alturaAcento);
    }
}
```

Exemplo: com uso de sobreposição de método

```
public class Test {
   public static void main(String args[])
   {

      MountainBike mb = new MountainBike(3, 100, 25);
      System.out.println(mb.toString());
   }
}
```

saída:

```
Marcha: 3
Velocidade: 100
Altura do acento: 25
```

Importante!

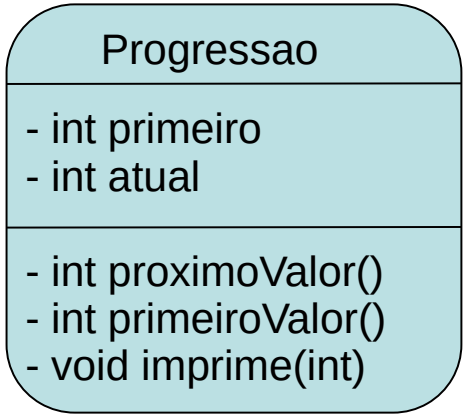
- **Superclasse default:** na ausência de qualquer superclasse explícita, toda classe em Java herda da classe **Object** (veja a classe Object)
- **Só pode haver uma superclasse:** uma superclasse pode ter várias subclasses, mas uma classe pode ter aenas uma superclasse. Java não suporta herança múltipla com classes... apenas com o recurso de interfaces!
- **Classes *final***: não podem ser herdadas. Ex.: `final class Aluno {...}`
- **Herança de construtores:** uma subclasse herda todos os membros de uma superclasse, menos o construtor (que não são membros), mas eles podem ser invocados com o uso de ***super***.
- **Herança de membros privados:** uma subclasse não herda membros privados. No entando, eles podem acessar membros privados de superclasses por meio dos métodos públicos (ou *protected*) herdados.

Polimorfismo

- Técnica que indica que uma mesma ação pode ser realizada de diferentes formas.
- Em Java há dois tipos de polimorfismo:
  - **De tempo de compilação (compile-time):** por meio de sobrecarga (overloading) de métodos.
  - **De tempo de execução (runtime):** por meio de sobreposição (overriding) de métodos.
- Pelo mecanismo de herança, o sentido de polimorfismo se firma no conceito de que uma superclasse pode assumir diferentes formas, por meio da sobreposição de métodos.

Exemplo

Progressão numérica




```
public class Progressao {
  protected int primeiro;
  protected int atual;

  Progressao() { primeiro = atual = 0; }

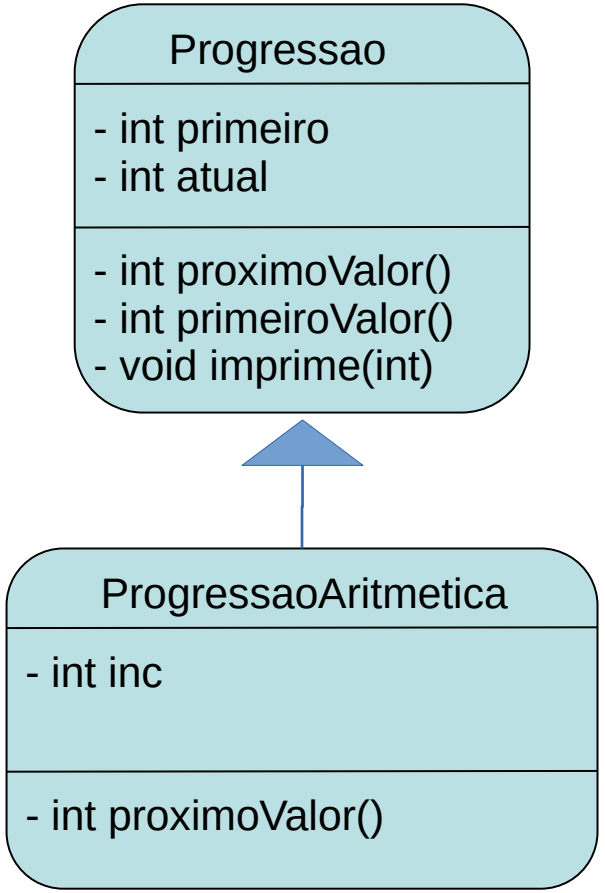
  public int proximoValor() {
    return ++atual;
  }

  public int primeiroValor() {
    atual = primeiro;
    return atual;
  }

  public void imprime(int n) {
    System.out.print(primeiroValor());
    for(int i=1; i<n; i++)
      System.out.print(" "+proximoValor());
    System.out.println();
  }
}
```

Exemplo

Progressão aritmética




```
public class ProgressaoAritmetica
extends Progressao
{
  protected int inc;

  public ProgressaoAritmetica(){
    this(1);
  }

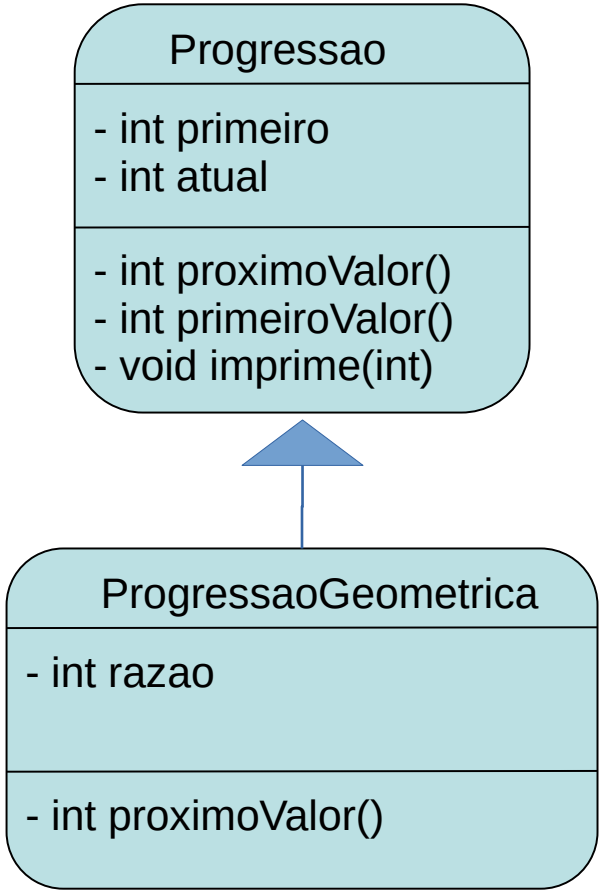
  public ProgressaoAritmetica(int inc){
    this.inc = inc;
  }

  @Overrride
  public int proximoValor() {
    atual += inc;
    return atual;
  }

}
```

Exemplo

Progressão geométrica




```
public class ProgressaoGeometrica
extends Progressao
{
  protected int razao;

  public ProgressaoGeometrica(){
    this(2);
  }

  public ProgressaoGeometrica(int razao){
    this.razao = razao;
    primeiro = 1;
    atual = primeiro;
  }

  @Overrride
  public int proximoValor() {
    atual *= razao;
    return atual;
  }

}
```

Exemplo

Testes:

```
class Main {
  public static void main(String[] args) {
    ProgressaoAritmetica progA = new ProgressaoAritmetica();
    progA.imprime(10);
  }
}
```

Saída: 0 1 2 3 4 5 6 7 8 9

Exemplo

Testes:

```
class Main {
  public static void main(String[] args) {
    ProgressaoGeometrica progG = new ProgressaoGeometrica();
    progG.imprime(10);
  }
}
```

Saída: 1 2 4 8 16 32 64 128 256 512

Exemplo

Testes:

```
class Main {
  public static void imprime(Progressao p){
    p.imprime(10);
  }

  public static void main(String[] args) {
    imprime(new ProgressaoAritmetica());
    imprime(new ProgressaoGeometrica());
    imprime(new ProgressaoGeometrica(3));
  }
}
```

Saída:

0 1 2 3 4 5 6 7 8 9
1 2 4 8 16 32 64 128 256 512
1 3 9 27 81 243 729 2187 6561 19683

Exemplo

Testes:

```
class Main {
  public static void imprime(Progressao p){
    p.imprime(10);
  }

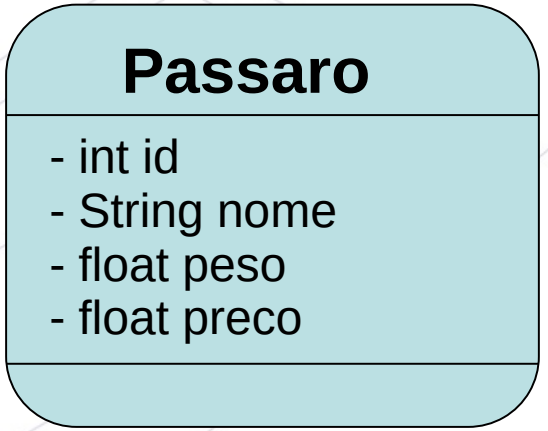
  public static void main(String[] args) {
    imprime(new ProgressaoAritmetica());
    imprime(new ProgressaoGeometrica());
    imprime(new ProgressaoGeometrica(3));
  }
}
```

upcasting

Run Time Polymorphism: também chamado de Dynamic Method Dispatch.

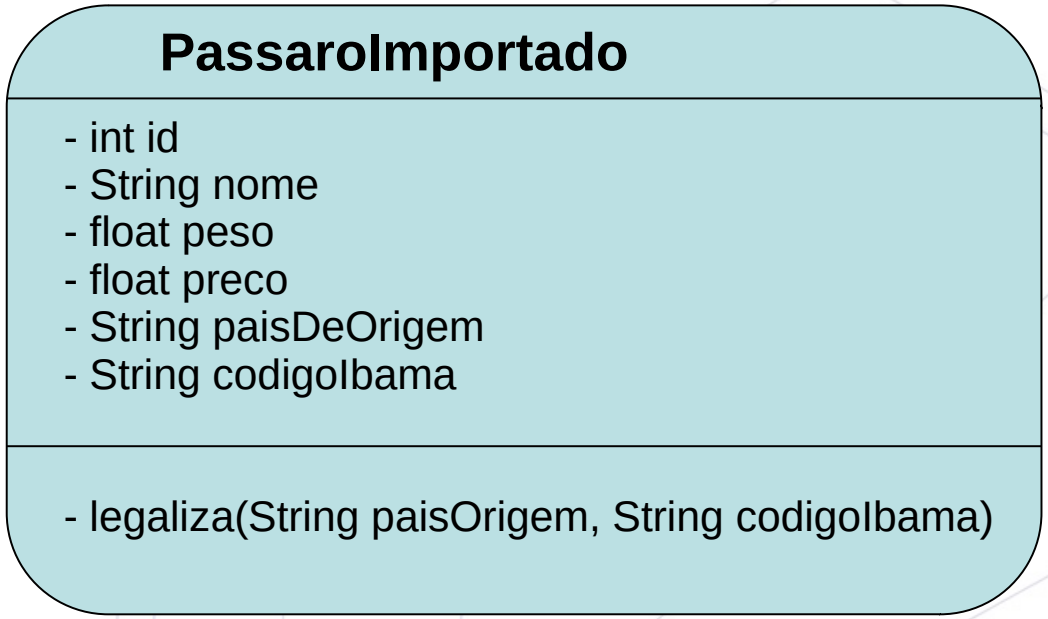
Exercício

1. PetShop: classes para um sistema de venda de pássaros em um petshop. O diagrama de classe para representar um pássaro é o seguinte:



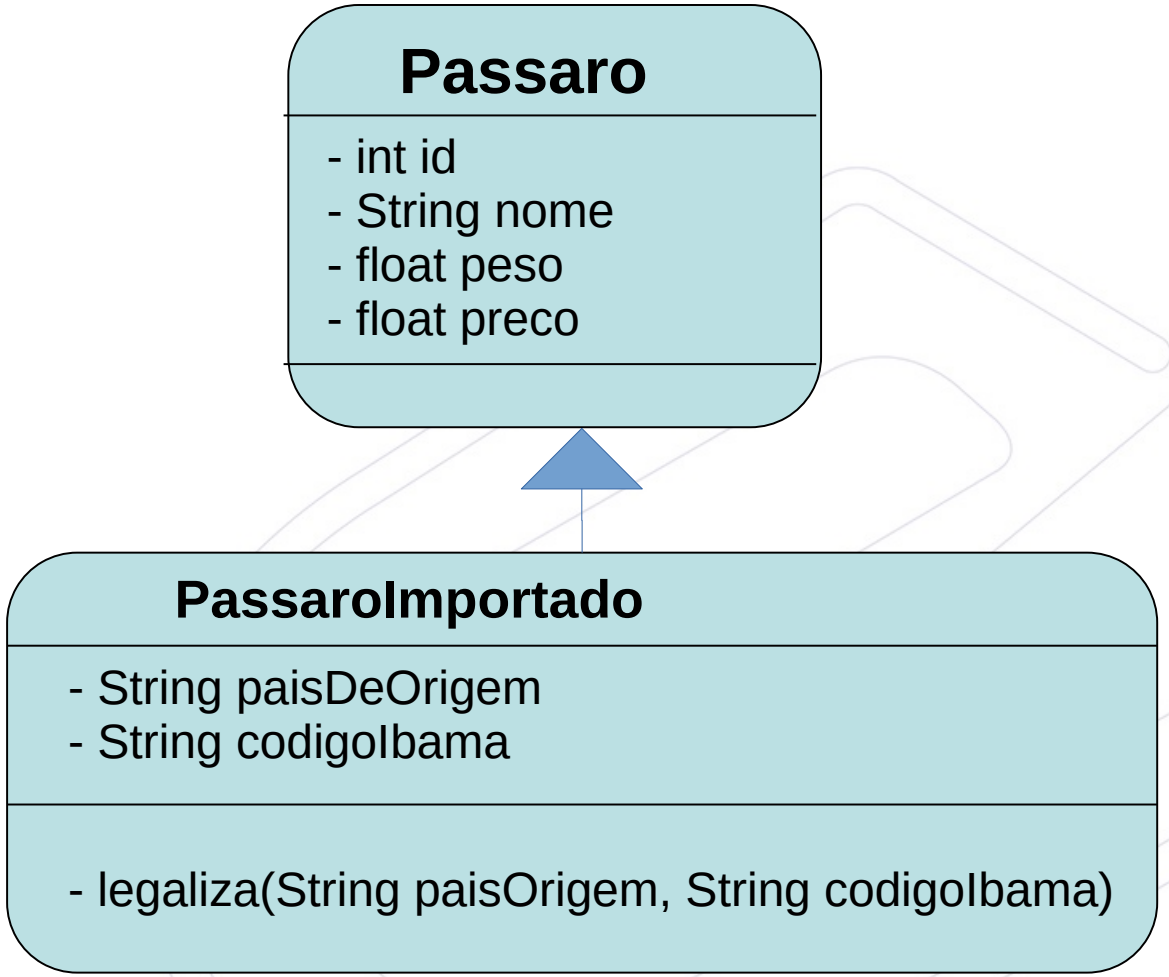
Exercício

2. PetShop: **Além de um pássaro comum, vendemos também pássaros de outros países.** É necessário saber sua origem e um código que o IBAMA usa para rastrear os animais que chegam do exterior.



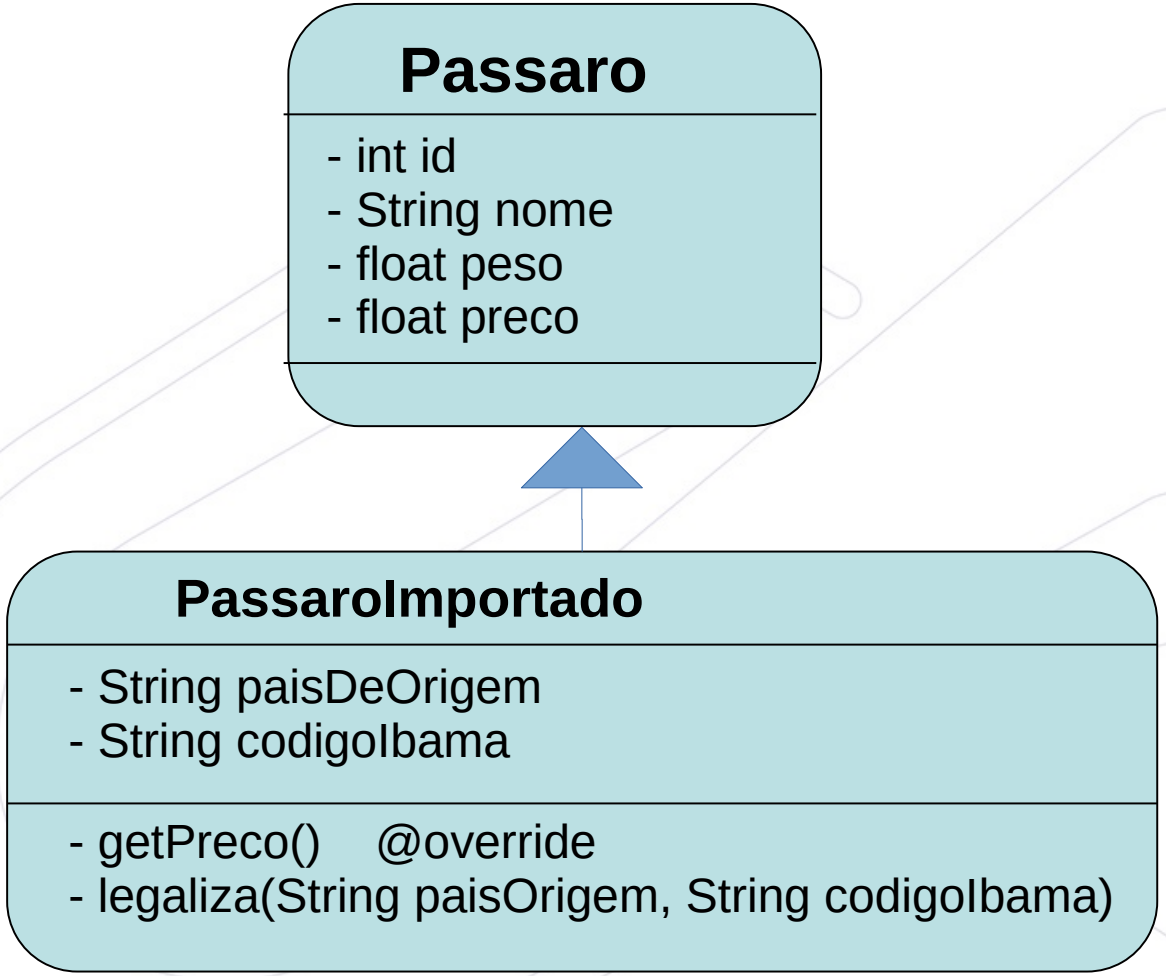
Exercício

2. PetShop: ...passaroImportado pode herdar de pássaro!



Exercício

3. PetShop: ...O valor de venda de um pássaro importado é 20% a mais que um pássaro naciona devido aos impostos cobrados pela importação.

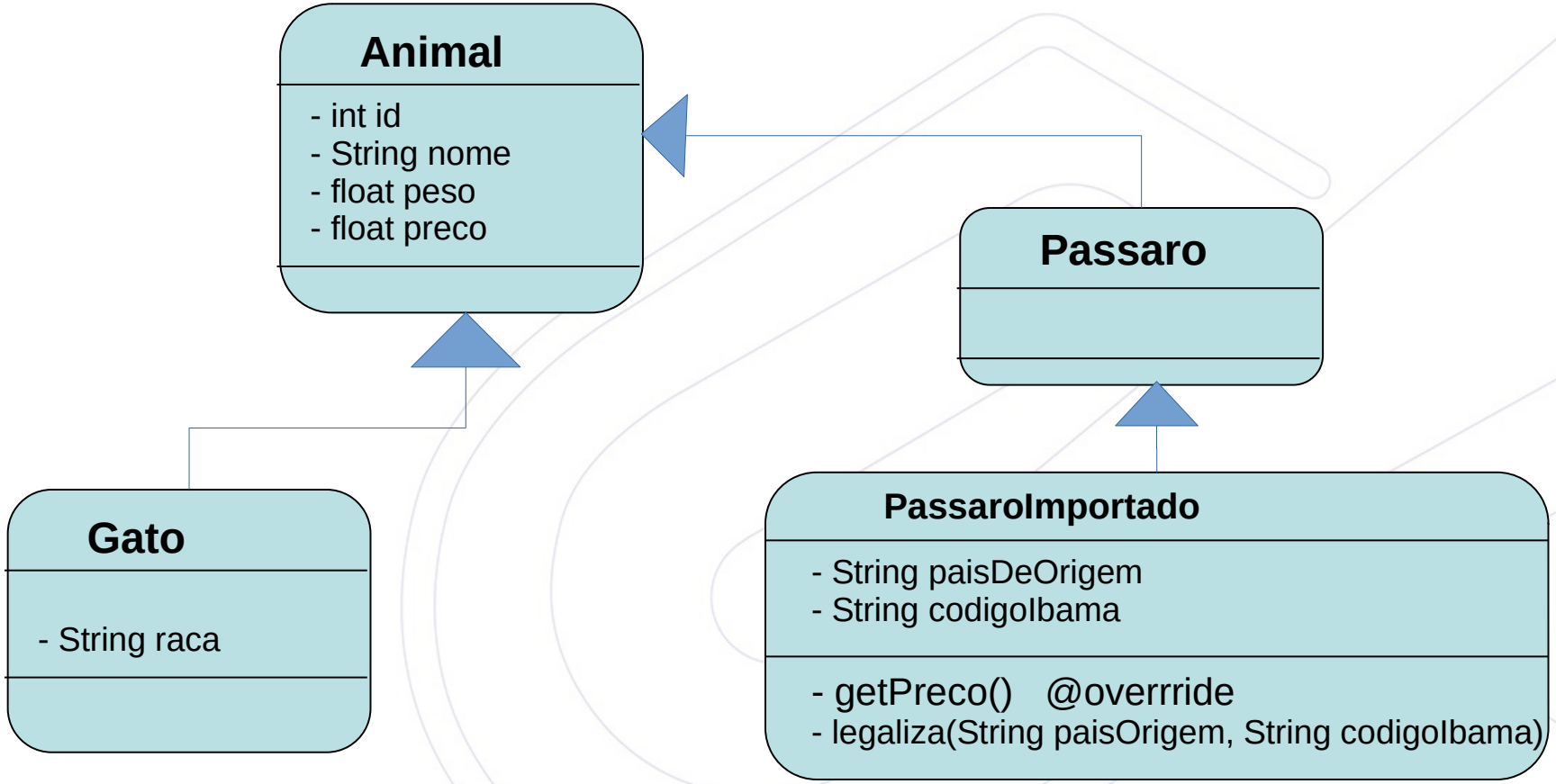


Exercício

4. PetShop: o petshop resolveu vender gatos também na qual possuem, além das informações que constam em um passaro nacional, a informação da raça do gato. Qual é a melhor solução para que o diagrama de classes faça sentido?

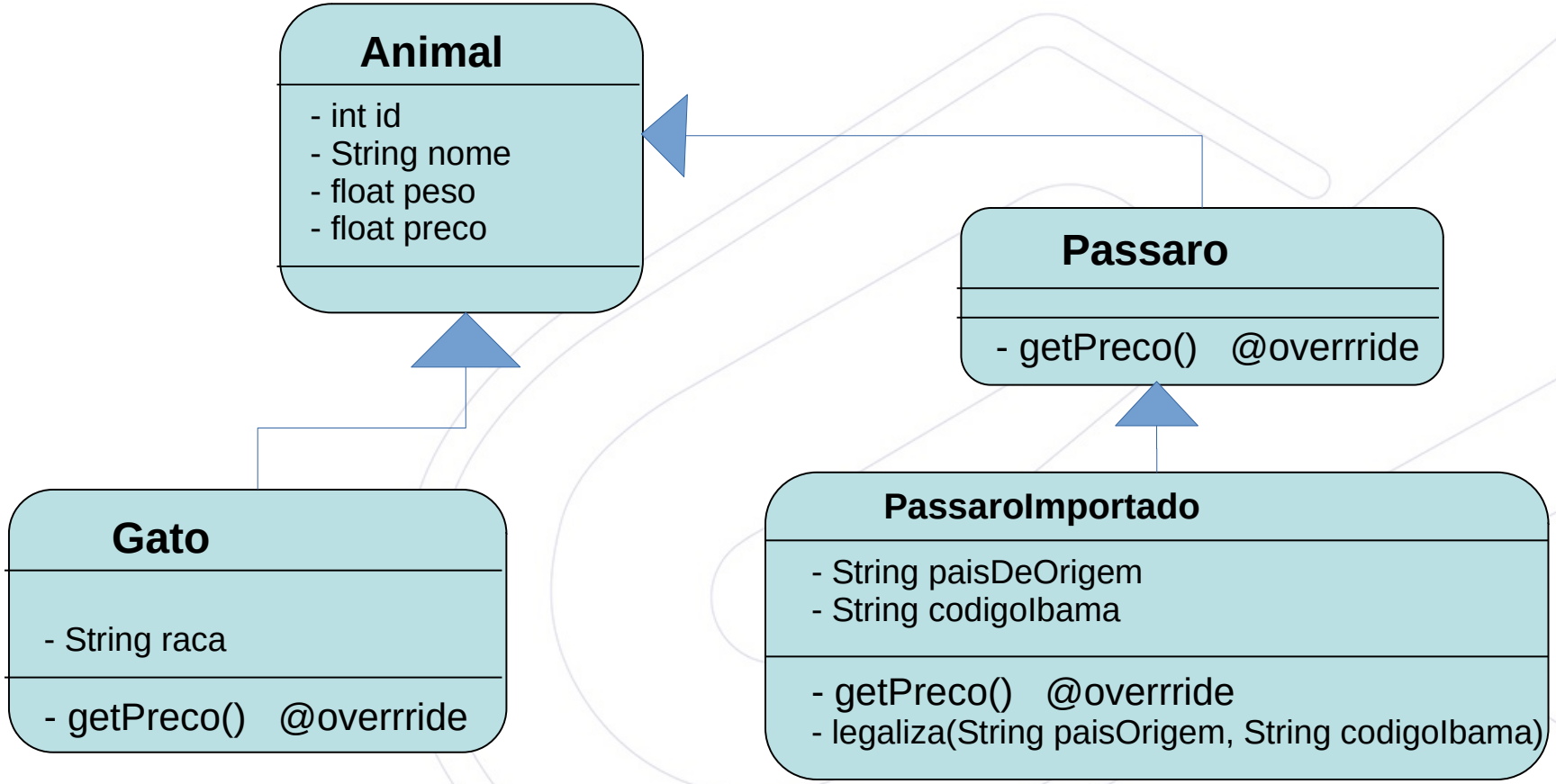
Exercício

4. PetShop: o petshop resolveu vender gatos também na qual possui, além das informações que constam em um passaro nacional, a informação da raça do gato. Qual é a melhor solução para o diagrama de classes faça sentido?



Exercício

4. PetShop: modifique a precificação para: Um pássaro nacional custa 5% a mais que preço de custo. Um pássaro importado custa 20% a mais que um nacional. Um gato custa 30% a mais que o preço de custo.



Exercício

3. PetShop: ...neste caso, se em outra parte do sistema tivéssemos um método que faça a venda de um animal, representado pelo método a seguir, qual é o preço apresentado na tela se for passada uma instância de um pássaro importado?

```
...
public void realizaVenda(Animal a) {
    ...
        System.out.println("Preço do animal: " + a.getPreco() );
        ...
}


...


PassaroImportado p = new PassaroImportado(125, "águia careca", 23, 10500);

realizaVenda(p);
```